

A New Approach to Sequence Comparison: Normalized Sequence Alignment

Abdullah N. Arslan, Ömer Egecioğlu and Pavel A. Pevzner

Bioinformatics, Vol.17, No.4, pp. 327-227, 2001

Abstract

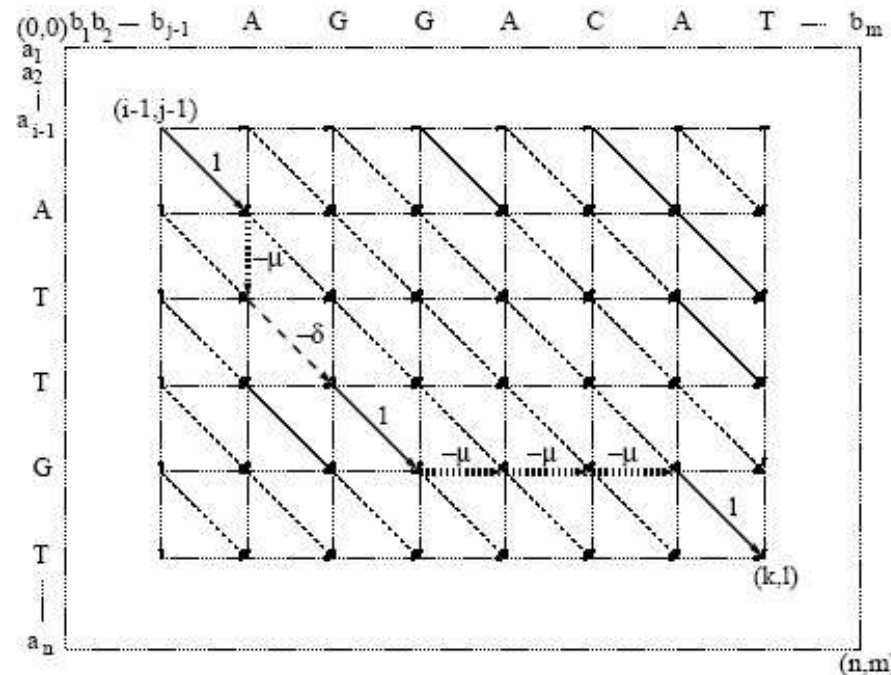
The Smith-Waterman algorithm for local sequence alignment is one of the most important techniques in computational molecular biology. This ingenious dynamic programming approach was designed to reveal the highly conserved fragments by discarding poorly conserved initial and terminal segments. However, the existing notion of local similarity has a serious flaw: it does not discard poorly conserved intermediate segments. The Smith-Waterman algorithm finds the local alignment with maximal *score* but it is unable to find local alignment with maximum *degree* of similarity (e.g., maximal percent of matches). Moreover, there is still no efficient algorithm that answers the following natural question: do two sequence share a (sufficiently long) fragment with more than 70% of similarity?

Abstract (Contd.)

As a result, the local alignment sometimes produces a mosaic of well-conserved fragments artificially connected by poorly-conserved or even unrelated fragments. This may lead to problem in comparison of long genomic sequences and comparative gene prediction as recently pointed out by Zhang et al., 1999 [33]. In this paper we propose a new sequence comparison algorithm (*normalized local alignment*) that reports the regions with maximum degree of similarity. The algorithm is based on fractional programming and its running time is $O(n^2 \log n)$. In practice, normalized local alignment is only 3-5 times slower than the standard Smith-Waterman algorithm.

The Problem and Definitions

- The Problem: to maximize $s(I, J)/(|I| + |J| + L)$ for a given parameter L . (Why L ?)
- (x, y, z) is an *alignment vector* for $a_i \cdots a_k$ and $b_j \cdots b_l$, if there is an alignment path with x matches, y mismatches, and z indels.
- A match has a score of 1, a mismatch is δ , and a gap penalty is μ .



Definitions (Contd.)

- $SCORE(x, y, z) = x - \delta y - \mu z$
- $AV_{i,j,k,l}(a, b) = \{ (x, y, z) | (x, y, z) \text{ is an alignment vector for } a_i \cdots a_k \text{ and } b_j \cdots b_k \}$
- $LA_{\delta,\mu}(a, b) : \text{maximize } x - \delta y - \mu z$
s.t. $(x, y, z) \in AV(a, b)$
- $NLA_{\delta,\mu,L}(a, b) : \text{maximize } \frac{x - \delta y - \mu z}{2x + 2y + z + L}$
s.t. $(x, y, z) \in AV(a, b)$
- The parametric local alignment problem:
 $LA_{\delta,\mu,L}(\lambda)(a, b) :$
 $\text{maximize } x - \delta y - \mu z - \lambda(2x + 2y + z + L)$
s.t. $(x, y, z) \in AV(a, b)$

Why L and $NLA^* < 1/2$

- Two sequences I, J and $I = J$ (best case).
- $L = 0$
 - ▶ $|I| = 1 \Rightarrow NLA^* = \frac{1}{2}$
 - ▶ $|I| = 100 \Rightarrow NLA^* = \frac{100}{200} = \frac{1}{2}$
- $L = 100$
 - ▶ $|I| = 1 \Rightarrow NLA^* = \frac{1}{2+100} = \frac{1}{102} \cong 0.01$
 - ▶ $|I| = 100 \Rightarrow NLA^* = \frac{100}{200+100} = \frac{100}{300} \cong 0.33$
- $NLA^* = \frac{|I|}{|I|+|J|+L} = \frac{|I|}{2|I|+L} < \frac{1}{2}$

PROPOSITION 1

PROPOSITION 1 *For a parameter λ ($< \frac{1}{2}$), the optimum value $LA^*(\lambda)$ of the parametric LA problem can be formulated in terms of the optimum value LA^* of an LA problem.*

PROOF. The objective of the parametric problem is

$$\begin{aligned} LA^*(\lambda) &= \max\{(1 - 2\lambda)x - (\delta + 2\lambda)y - (\mu + \lambda)z - \lambda L\} \\ &= (1 - 2\lambda) \max\left\{x - \frac{\delta + 2\lambda}{1 - 2\lambda}y - \frac{\mu + \lambda}{1 - 2\lambda}z\right\} - \mu L \\ &= (1 - 2\lambda) LA_{\delta', \mu'}^*(a, b) - \lambda L \end{aligned}$$

where $\delta' = \frac{\delta + 2\lambda}{1 - 2\lambda}$, $\mu' = \frac{\mu + \lambda}{1 - 2\lambda}$

Algorithm

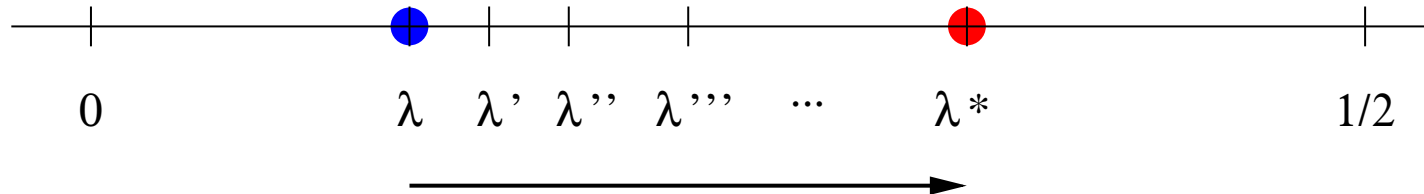
- For any alignment vector (x', y', z') ,
$$\lambda = \frac{x' - \delta y' - \mu z'}{2x' + 2y' + z' + L} < \frac{1}{2}$$

(Why $\frac{1}{2}$?)
- Dinkelbach's and Megiddo's methods can be used to solve NLA .
- $\lambda = NLA^*$ iff $LA^*(\lambda) = 0$

Algorithm (Contd.)

1. Pick a λ . (How to pick a λ ?)
2. Using Prop.1, solve $LA(\lambda)$ and obtain an optimal alignment vector (x, y, z) .
3. If (x, y, z) has the optimum normalized score, then (x, y, z) is the answer; else goto step 1.

Dinkelbach Algorithm



Algorithm Dinkelbach

Pick an arbitrary alignment vector (x, y, z) in $AV(a, b)$,

$$\lambda^* \leftarrow \frac{x - \delta y - \mu z}{2x + 2y + z + L}$$

Repeat

$$\lambda \leftarrow \lambda^*$$

Using Prop.1, solve $LA(\lambda)$ and
obtain an optimal alignment vector (x, y, z)

$$\lambda^* \leftarrow \frac{x - \delta y - \mu z}{2x + 2y + z + L}$$

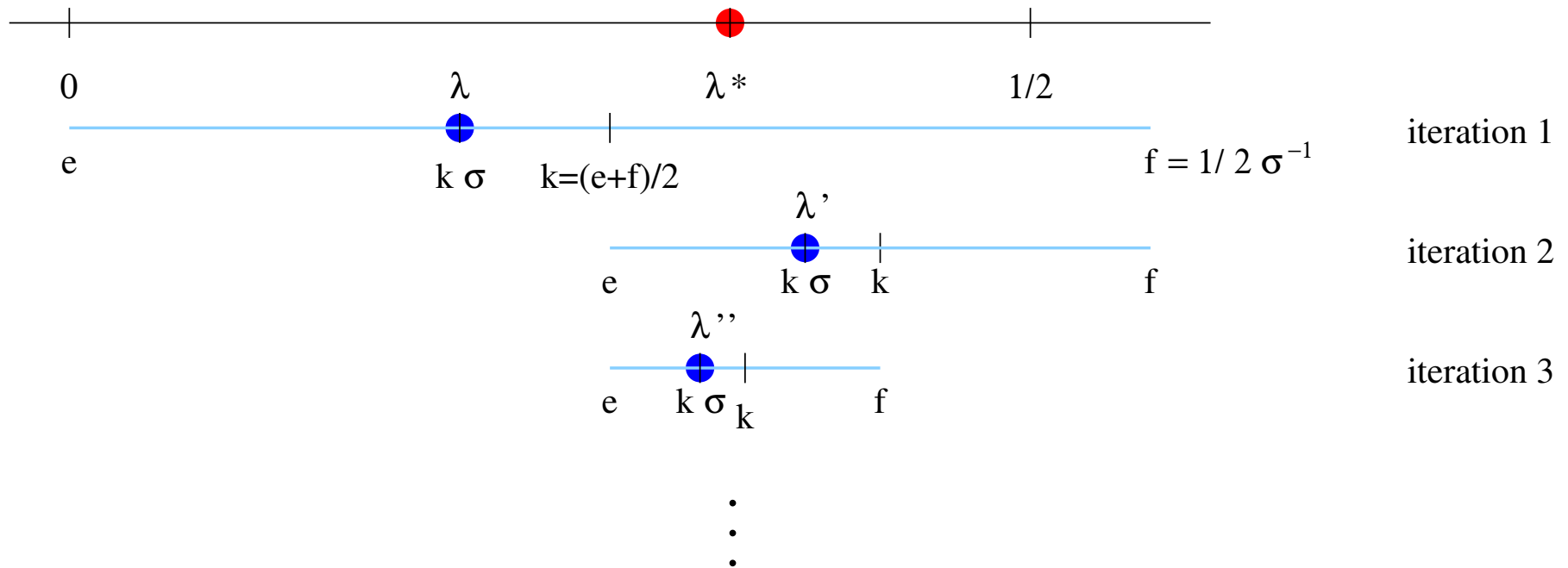
Until $\lambda^* = \lambda$

Return(λ^*)

Figure 4: Dinkelbach algorithm for NLA .

RationalNLA

An efficient search (*a binary search*) for the optimum value if the scores are rational.



RationalNLA (Contd.)

Algorithm RationalNLA

$\sigma \leftarrow \frac{1}{qs(m+n+L)^2}$ where $\delta = \frac{p}{q}$, and $\mu = \frac{r}{s}$ (This is the gap lower bound)

$[e, f] \leftarrow [0, \frac{1}{2}qs(m+n+L)^2]$

While $(e+1 < f)$ do

$k \leftarrow \lfloor (e+f)/2 \rfloor$

 Using Prop.1, solve $LA(k\sigma)$ and obtain an optimal alignment vector (x, y, z)

$v \leftarrow \frac{x - \delta y - \mu z}{2x + 2y + z + L}$

 if $v = 0$ then return($k\sigma$)

 else if $v < 0$ then $f \leftarrow k$

 else $e \leftarrow k$

End {while}

Using Prop.1, solve $LA(f\sigma)$ and obtain an optimal alignment vector (x, y, z)

Return($\frac{x - \delta y - \mu z}{2x + 2y + z + L}$)

Figure 5: NLA algorithm RationalNLA for rational scores.

The Time Complexity

- Dinkelbach algorithm for NLA:
the number of iterations $\times$ the time complexity of the Smith-Waterman algorithm.
- RationalNLA: $O(nm \log n)$

Thanks

Thanks

Date: Sep. 03, 2004

Created by: Liang-Tai Wei